Build a Production SaaS Platform From Scratch

A rebuild-from-zero course in architecture, engineering practice, and the decisions behind them

Perezosoft Platform · generated 2026-07-08

Contents

Preface

What this book is

This is a course disguised as a book. Over ten parts and fifty lessons, you will build — by typing every hand-written line yourself — a production-grade, multi-tenant SaaS platform: custom JWT authentication with passwordless sign-in, OAuth, and MFA; tenant isolation enforced by the type system *and* by the database; billing with Stripe; reliable asynchronous work via a transactional outbox; role-based access control; file storage; GDPR export and erasure; a public API; outbound webhooks; an admin back-office; full observability — and at the end you will deploy it to a real host behind a real CI/CD pipeline that you also built.

The code is real. Every excerpt in this book is drawn from a working reference platform (Perezosoft Platform), not invented for the page. A coverage gate — a script, in the same spirit as the tests you'll write — maps every file of that platform to the lesson that builds it, so nothing is skipped and nothing appears out of thin air.

But the code is not the point. The point is the two habits that separate an architect from a very fast typist:

1. **Making invariants structural.** Not "remember to filter by tenant" but "a query *cannot* forget to filter by tenant." Not "please don't call this API in feature code" but "the build fails if you do." You will meet this move dozens of times — in query filters, interceptors, marker interfaces, architecture tests, CI gates, database policies — until reaching for it is a reflex.
1. **Recording decisions.** Every lesson ends with an *Architecture Decision* box: the fork that was faced, the option chosen, the options rejected, and the price paid. You will write your own decision log from lesson 0.1 onward, before any code exists. Six months from now, "why is it like this?" will have an answer.

Who this is for

You are comfortable in C# — classes, interfaces, async/await, LINQ hold no fear — and you can find your way around a terminal and git. You have probably built applications, perhaps professionally, but you want the *architecture* to stop feeling like folklore: why layers, why seams, why tests first, why all these patterns with strange names, and — above all — when *not* to use them.

You do not need prior experience with multi-tenancy, EF Core internals, Stripe, Docker, OAuth, or GitHub Actions. Each is taught when the platform needs it, motivated by a concrete problem you will feel before you solve it.

What this book is not

Honesty up front: this is a course in **systems design and engineering practice**, not computational problem-solving. You will find no algorithm puzzles and almost no performance engineering here — no profilers, and caching is deliberately deferred (a recorded decision you'll read about). If you want to get better at data structures, this is the

wrong book; if you want to get better at *building software that survives contact with production and with other people*, it is the right one.

The rhythm

Every lesson follows the same beat, and the beat is the curriculum:

1. **Motivate** — a concrete problem, ideally demonstrated by a failing or *leaking* test.
2. **Red** — write the test that pins the behavior you want. It fails.
3. **Green** — write the minimum code to pass it.
4. **Refactor & harden** — extract the seam; add the guardrail that makes the mistake impossible to repeat.

1. **Run it** — commands and expected output; every lesson ends with the app working.
2. **Architecture Decision** — the fork, the choice, the road not taken.
3. **Checkpoint** — a git tag; what you should now be able to do.

Test-driven development is not a chapter in this book; it is the loop you will live roughly fifty times, exactly as the reference platform was actually built.

How to use this book

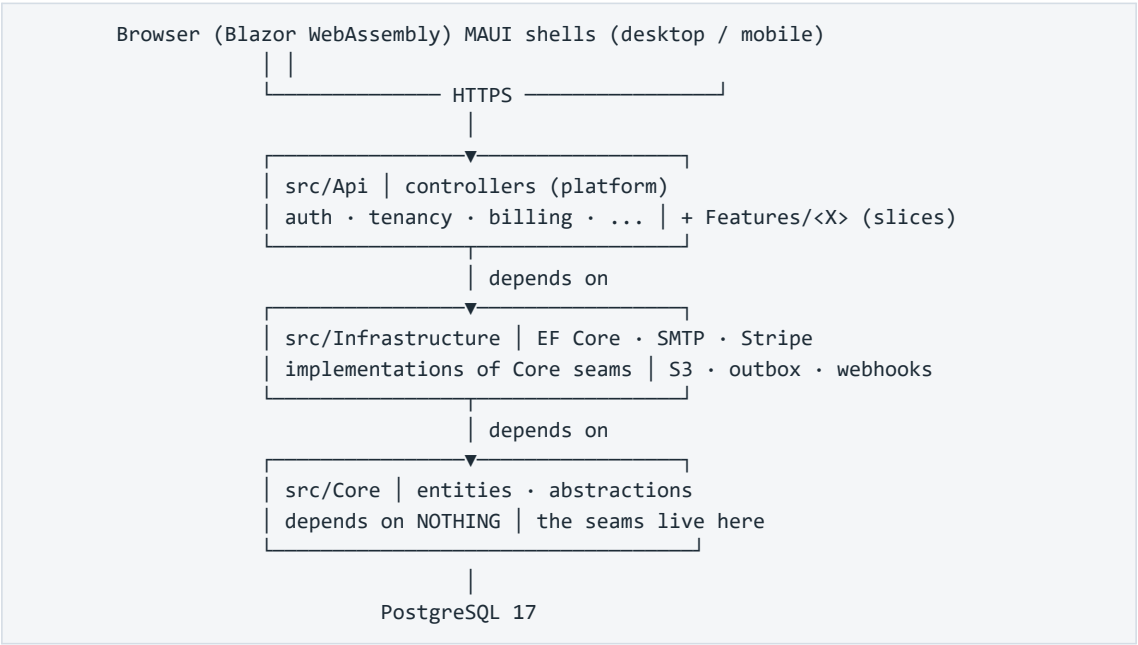
The companion repository. Alongside the book there is a companion repo built in teaching order, with one tagged checkpoint per lesson (`lesson-1.3`, `lesson-2.6`, …). Type everything yourself — that is the course — but when you are stuck, `git diff` your work against the checkpoint, or check it out and keep moving. Falling behind the typing is recoverable; skipping the understanding is not.

Don't skip Part 0. Lesson 0.2 pins your toolchain and infrastructure so that the other forty-nine lessons behave identically on your machine. Most abandoned courses die at "works on my machine"; we kill that failure mode first.

When a lesson cites a rule (R1–R35) or an ADR, that is a pointer into the reference platform's own governance documents — the machine-enforced invariants and the decision log. You are not just building an app; you are building the *system that keeps the app honest*, and those citations show you where each piece of it came from.

Reading order is dependency order. Later lessons lean on earlier seams without re-explaining them. If Part 5 feels steep, the fix is usually two parts back, not a re-read of Part 5.

The platform at a glance



One sentence to keep for the whole book: **app data belongs to the tenant, not the user** — and by the end of Part 2 you will have made it impossible, at three separate layers, for any line of feature code to violate that sentence.

Part 0 — Orientation

Lesson 0.1 — The mental model & the decision record

> **Part 0 · Orientation.** No code in this lesson — deliberately. You're about to type several hundred files over ~48 lessons. The two things that determine whether that makes you an architect or a very tired typist are (1) the mental model you hang each file on, and (2) the habit of recording *why* before *what*. Both are installed here.

Goal: you can draw the system from memory — its layers, its one non-negotiable invariant, and its unit of work — and you've written your project's first ADR.

Concepts & principles: multi-tenancy (tenant ≠ user); clean architecture's dependency rule; vertical slices vs. horizontal layers; Architecture Decision Records.

Maps to: ADR-C1/C2/C3, ADR-004 · `docs/DECISIONS.md`, `docs/WAYS_OF_WORKING.md` in the reference repo.

---

1. What you are going to build

A production-grade, multi-tenant SaaS **platform**: custom JWT auth (passwordless + OAuth + MFA), tenant isolation enforced by the type system, billing with Stripe, reliable async via a transactional outbox, RBAC, file storage, GDPR export/erasure, a public API, outbound webhooks, an admin back-office, observability — deployed to a real host at the end. Not a to-do app. The kind of chassis a real product bolts features onto.

You will build it **test-first, in vertical slices**, and every architectural rule will be enforced by a test you wrote — not by a comment hoping to be read.

2. The four ideas everything else hangs on

Idea 1 — Tenant ≠ user

The single most important sentence in this course: **app data belongs to the tenant, not the user.** A *tenant* is the paying unit — a household, a team, a company. Users are members of a tenant; they come and go, and the tenant's data stays. Get this backwards and you'll design tables, queries, and permissions around the wrong axis — and unwinding that later is a rewrite, not a refactor.

One consequence you'll meet in Lesson 2.6 and never stop meeting: every read and write of app data must be scoped to the current tenant, and that scoping must be **impossible to forget**, not merely encouraged.

Idea 2 — The dependency rule (clean architecture, minus the ceremony)

Three production layers, dependencies pointing strictly inward:

```
Api ──► Infrastructure ──► Core
(HTTP, auth, (EF Core, SMTP, (entities, abstractions —
 controllers, Stripe, S3, the depends on NOTHING)
 features) implementations)
```

`Core` holds the entities and the *seams* — small interfaces like `IEmailSender`, `IFileStorage`, `IBillingProvider`. `Infrastructure` implements them with real technology (MailKit, S3, Stripe). `Api` orchestrates. The payoff is swappability where it matters (Mailpit in dev, Brevo in prod — same seam) and testability everywhere. The UI (Blazor WebAssembly) is *just another API client* — it never touches the database.

Idea 3 — Clean platform + vertical slices (the hybrid)

Pure layered architecture scatters one feature across four folders. Pure feature-folders duplicate the platform in every slice. This codebase does both, deliberately, and the split is the point:

- The **platform** — auth, tenancy, billing, email, persistence — is the durable chassis,

organized in clean layers. Built once, reused by everything.

- Each **app feature** lives in ONE folder (`src/Api/Features/<Feature>/`) containing its

endpoints, handler, DTOs, and data contributor. Features *reuse* the platform; they never reach into each other.

When you wonder "where does this file go?", the question to ask is: *chassis or feature?*

Idea 4 — Vertical slices as the unit of work

Every increment of work cuts through all layers — DB to UI — and leaves the app working. You'll never build "all the tables" then "all the endpoints." Each lesson in this course is itself a slice: it ends with a green build, passing tests, and a tagged checkpoint.

3. The decision record — the habit that makes it stick

Here's what separates a codebase you can *maintain* from one you can only *fear*: six months from now, the question is never "what does this code do?" (you can read it), it's **"why is it like this — and is it safe to change?"**

An **Architecture Decision Record** answers that in ~10 lines, written at the moment of decision:

```
## ADR-002 — Custom JWT + rotating refresh tokens (not ASP.NET Core Identity)

**Date:** 2026-06-XX · **Status:** Accepted

**Context:** We need auth for web + native clients with passwordless and OAuth.
Identity brings cookie-centric defaults, schema we don't control, and hides the
token lifecycle we need to reason about (MFA step-up, rotation, native flows).

**Decision:** Issue our own short-lived JWT access tokens + rotating refresh
tokens; store only token hashes.

**Consequences:** We own token security (rotation, reuse detection — must test
it ourselves). In exchange: full control of claims (tenant_id drives data
isolation), one auth model across web/native, no framework fight when MFA lands.
```

Note what an ADR is *not*: not documentation of how the code works (the code shows that), and never deleted when wrong — a reversed decision gets a **new dated ADR** superseding the old one. The history of your reasoning is itself an asset.

The reference repo runs on this: 20 app ADRs + amendments in `docs/DECISIONS.md`. Every lesson ahead ends with an **Architecture Decision box** — the fork faced, the option chosen, the roads not taken. By Part 4 you should be predicting them before reading them; that reflex *is* the architectural skill this course exists to build.

4. Exercise — write your ADR-000 and ADR-001

Create your project directory (empty — the solution comes in Lesson 1.1) and start the decision log:

```
mkdir my-saas && cd my-saas && git init
mkdir docs && $EDITOR docs/DECISIONS.md
```

Write two ADRs yourself — don't copy:

1 **ADR-000 — Why I'm building this.** Context (what you want to learn), decision (build the platform from scratch, test-first), consequences (weeks of effort; deep ownership of every line).

1 **ADR-001 — Local secrets live in `.env`, never in the repo.** You haven't built the loading mechanism yet (that's 0.2 and 1.4) — which is exactly the point: **the decision precedes the code.** Context: secrets in committed files leak (git history is forever). Decision: a gitignored `.env` is the single local source of truth; a committed `.env.example` documents every key; production uses real environment variables. Consequences: one more setup step per machine; zero secrets in history — enforced by a scanner you'll add in the next lesson.

5. Checkpoint

```
git add docs/DECISIONS.md && git commit -m "docs: ADR-000 and ADR-001 — the decision log starts before the code"
git tag lesson-0.1
```

You should now be able to: state the tenant≠user invariant and its consequence for every future query; draw the three layers and their dependency direction; say which of the two organizational modes (chassis vs. feature folder) a given file belongs to; and write an ADR that captures context → decision → consequences in under a page.

Next: *Lesson 0.2 — A reproducible machine*, where your ADR-001 gets its mechanical teeth: `.env.example`, docker-compose, and a secret scanner.

Lesson 0.2 — A reproducible machine

> **Part 0 · Orientation.** Most from-scratch courses die at "install PostgreSQL." This lesson exists so yours can't. At the end, your machine runs the exact infrastructure the app will use for the next 47 lessons — pinned, containerized, verifiable with three commands — and your ADR-001 (secrets never in the repo) is enforced by machinery, not memory.

Goal: `docker compose up -d` gives you a healthy Postgres 17 and a mail trap; `dotnet --version` prints the pinned SDK; a secret can't reach a commit.

Concepts & principles: pinned toolchains ("latest stable, never previews" — and never "whatever was on the machine"); infrastructure-as-code for dev; the `.env` contract (ADR-001); fail-loud health checks; defense-in-depth for secrets.

Maps to: ADR-001, ADR-C13 · Foundation theme R20–R22 (the config contract this enables) · repo: `docker-compose.yml`, `.env.example`, `.gitignore`, `.gitleaks.toml`.

Prerequisites: 0.1 (your repo exists and ADR-001 is written).

---

1. Motivate — "works on my machine" is an architecture failure

Two developers clone the same repo. One has Postgres 15 installed as a Windows service from a project two jobs ago; the other has nothing. One gets a subtle JSON-function bug that doesn't reproduce; the other gets a connection refused. Neither failure is *in the code* — and that's the point: **your dev environment is part of your architecture**, and undeclared dependencies are its silent killers.

The fix is the same one you'll apply to code all course long: make the implicit explicit, and make violations loud.

2. Install the pinned toolchain

Three tools go on the machine itself; everything else runs in containers:

Tool	Version	Check
.NET SDK	**10.0.3xx** (the line this course pins)	`dotnet --version`
Docker Desktop / Engine	current stable	`docker version`
EF Core tooling	current stable	`dotnet tool install --global dotnet-ef` (first used in 1.3)

Pin the SDK **in the repo**, not in a README nobody reads — create `global.json`:

```
{
  "sdk": { "version": "10.0.301", "rollForward": "latestFeature" }
}
```

Now a machine with the wrong SDK fails at `dotnet build` with an explicit message instead of failing three lessons later with an inscrutable one. This is the course's recurring move — *fail*

loud and early — applied for the first time, to the toolchain itself.

3. Infrastructure as one file — docker-compose

The app needs PostgreSQL and — less obviously — an **SMTP trap**. From Lesson 2.3 onward the platform sends real email (magic links, OTPs, invitations), and you need somewhere for it to land in dev that isn't a real inbox. Mailpit accepts SMTP on one port and shows every message in a web UI on another — and, critically for Lesson 3.6, exposes an **HTTP API your tests will query** to fish out OTP codes. Two infrastructure decisions, one committed file — create `docker-compose.yml`:

```
services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME:-dev_db}
      POSTGRES_USER: ${DB_USER:-dev}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-devpassword}
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-dev} -d ${DB_NAME:-dev_db}"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 10s

  mail:
    image: axllent/mailpit:latest
    restart: unless-stopped
    ports:
      - "${MAIL_SMTP_PORT:-1025}:1025" # SMTP — the app sends here
      - "${MAIL_UI_PORT:-8025}:8025" # Web UI + API — you and your tests read here
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider http://localhost:8025 || exit 1"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 5s

volumes:
  db_data:
```

Read it as a set of decisions, because each line is one:

- **`postgres:17`** — a pinned major version, mirroring `global.json`'s job for the SDK.

"Latest stable, never previews" also means *never unpinned*.

- **`${DB_PORT:-5432}`** — every value is overridable via environment, with a working

default. A developer whose port 5432 is squatted by that old Windows service sets `DB_PORT=5433` in `.env` and moves on. (The reference repo defaults to 5433 for exactly that reason — port collisions are the #1 setup killer.)

- **`healthcheck`** — "container started" ≠ "Postgres accepting connections." Health

checks make readiness explicit, and anything that later `depends_on` these services waits for *healthy*, not merely *running*. You'll re-meet this exact idea as the API's own `/health` endpoint in Lesson 4.5.

- **`volumes: db_data`** — your data survives `docker compose down`. (Destroying it becomes a deliberate act: `docker compose down -v`.)

4. The `.env` contract — ADR-001 gets teeth

Your ADR-001 said: secrets live in a gitignored `.env`; a committed `.env.example` documents every key. Build both halves now.

`.gitignore` (start; it grows with the solution in 1.1):

```
.env
bin/
obj/
storage/
```

`.env.example` — **committed**, and it is *documentation with a job*:

```
# Copy this file to .env and fill in your values. .env is gitignored — never commit it.
# Single source of local-dev config + secrets: read by docker-compose (containers) AND
# by the API via DotNetEnv (from Lesson 1.4). See docs/DECISIONS.md ADR-001.

# ── Containers (docker-compose) ──────────────────────────
DB_NAME=dev_db
DB_USER=dev
DB_PASSWORD=devpassword
DB_PORT=5433

# Mailpit (SMTP trap)
MAIL_SMTP_PORT=1025
MAIL_UI_PORT=8025
```

Then `cp .env.example .env` and confirm `git status` does **not** list `.env`.

Notice the shape of the contract: *every* key the system reads appears here with an inline comment saying what it does, whether it's required, and what happens when unset. In Lesson 1.4 you'll write a test (`DocAndConfigSyncTests`) that **fails the build when a config key exists in code but not in this file** — turning "please document your config" from a review nag into a compile-adjacent guarantee (R20). The habit starts now.

Docker-compose reads `.env` automatically. So one file drives both the containers and — once DotNetEnv arrives in 1.4 — the app. Two consumers, one source of truth, zero drift.

The second layer: a secret scanner

`.gitignore` is advisory — it does nothing if someone hardcodes a connection string in a `.cs` file. Add gitleaks config, `.gitleaks.toml`:

```
# Secret scanning — the backstop behind ADR-001. Default rules + our allowlist.
[extend]
useDefault = true

[allowlist]
description = "Dev-only placeholder values that look like secrets but aren't"
paths = ['''\.env\.example''']
```

Run it locally (`gitleaks detect`) whenever you like; in Lesson 8.3 it becomes a CI gate. Layers, again: `.gitignore` prevents the accident, gitleaks catches what slips past, CI makes it policy. No single layer is trusted alone — the same defense-in-depth shape you'll apply to tenancy (filter + interceptor + arch test) from Part 2 on.

5. Run it — verify everything

```
docker compose up -d
docker compose ps # both services: status "healthy" (wait a few seconds)
dotnet --version # 10.0.3xx — matches global.json
docker compose exec db psql -U dev -d dev_db -c "select version();" # PostgreSQL 17.x
```

Open http://localhost:8025 — Mailpit's empty inbox. In Lesson 2.4, your app's first magic-link email lands here; in 3.6, your Playwright tests read OTP codes out of it through its API.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: where does dev infrastructure live? (a) Installed natively per machine, (b) a shared cloud dev database, (c) containers defined in the repo.

Chosen: containers (ADR-C13). The infrastructure's *versions and topology* are code, reviewed like code; a new machine is productive in minutes; two projects with different Postgres majors coexist.

Rejected: native installs — unpinned, colliding, undocumented by nature. Shared cloud dev DB — couples every developer's iteration loop to a network and to each other's data (and costs money before the first feature exists).

The trade: Docker becomes a hard dev dependency. Accepted knowingly — it's also the test-isolation strategy (Testcontainers, Lesson 1.3) and the deployment unit (8.2), so the dependency pays three times.

7. Checkpoint

```
git add global.json docker-compose.yml .gitignore .gitleaks.toml .env.example
git commit -m "chore: reproducible dev environment — pinned SDK, compose infra, env
contract"
git tag lesson-0.2
```

You should now be able to: bring the full dev infrastructure up and down with one command; explain why the SDK and Postgres versions are pinned *in the repo* rather than in a README; state the `.env` / `.env.example` contract and the two layers that enforce it; and articulate why health checks matter before anything depends on these services.

Next: *Lesson 1.1 — Solution, projects & supply chain* — the .NET solution appears, with its dependencies pointing the right way and its packages locked.

Part 1 — The walking skeleton

Lesson 1.1 — Solution, projects & supply chain

> **Part 1 · The walking skeleton.** Nothing runs yet — that's next lesson. Here we lay out the solution so that the architecture from Lesson 0.1 isn't a diagram you promised to respect but a set of project references the compiler enforces. We also make a decision most projects defer until it hurts: exactly which versions of everything we depend on, pinned so a build today and a build in six months produce the same bytes.

Goal: a solution with every project in place, dependencies physically pointing inward, one central version table, and a locked, reproducible restore.

Concepts & principles: the dependency rule made mechanical; Central Package Management; lockfiles and supply-chain hygiene; warnings-as-errors from day one.

Maps to: ADR-C3/C4/C5 · R25–R27 · repo: `Perezosoft.slnx`, `Directory.Build.props`, `Directory.Packages.props`, `*/*.csproj`.

Prerequisites: 0.2 (pinned SDK; `dotnet --version` prints 10.0.3xx).

---

1. Motivate — the layout is the first enforcement

Every codebase claims to have an architecture. The honest question is: *what happens if someone violates it?* If the answer is "a reviewer might notice," you have a convention. If the answer is "it does not compile," you have a structure.

Project references are the cheapest structural enforcement .NET offers. When `Core` has no reference to `Infrastructure`, a domain entity *cannot* reach out to EF Core or SMTP — not "should not," cannot. The dependency rule from Lesson 0.1 stops being a poster on the wall the moment the reference graph encodes it. That's what we build in this lesson.

The second problem we solve is quieter and nastier. `dotnet add package` with no version pinned means your build depends on *when* you ran it. Two developers restore on different days and get different transitive graphs; CI builds an artifact subtly unlike the one you tested; and when a compromised package version hits NuGet — it has happened — nothing in your build even notices. Reproducibility isn't a nicety; it's the precondition for being able to say "this is what we shipped."

2. Create the skeleton

The solution uses the modern XML solution format (`.slnx` — human-readable, merge-friendly, supported by .NET 10 tooling):

```
dotnet new sln --format slnx -n Perezosoft
dotnet new classlib -o src/Core -n Perezosoft.Core
dotnet new classlib -o src/Infrastructure -n Perezosoft.Infrastructure
dotnet new web -o src/Api -n Perezosoft.Api
dotnet new blazorwasm -o src/Web -n Perezosoft.Web
dotnet new razorclasslib -o src/Shared.Ui -n Perezosoft.Shared.Ui
dotnet new xunit -o tests/Core.Tests -n Perezosoft.Core.Tests
dotnet new xunit -o tests/Api.Tests -n Perezosoft.Api.Tests
dotnet new nunit -o tests/E2E.Tests -n Perezosoft.E2E.Tests
dotnet sln add src/Core src/Infrastructure src/Api src/Web src/Shared.Ui \
              tests/Core.Tests tests/Api.Tests tests/E2E.Tests
```

Delete the template's sample classes (`Class1.cs`, the Blazor counter page can stay for now — it goes when the real UI arrives in 3.4). Then wire the references — this is the architecture, so read it slowly:

```
dotnet add src/Infrastructure reference src/Core # Infra implements Core seams
dotnet add src/Api reference src/Core src/Infrastructure # Api composes both
dotnet add src/Web reference src/Shared.Ui # Web hosts the shared UI
dotnet add tests/Core.Tests reference src/Core
dotnet add tests/Api.Tests reference src/Api src/Infrastructure src/Core
```

What's deliberately **absent** is the point: `Core` references *nothing* of ours. `Infrastructure` cannot see `Api`. `Web` cannot see `Core`, `Infrastructure`, or `Api` at all — the UI will talk to the API over HTTP like any other client (Golden Rule: the clean API boundary), and the compiler now holds it to that. Try it: add `using Perezosoft.Infrastructure;` to a file in `Core` — it does not build. That error message is the architecture working.

Each project file stays nearly empty. Here is the real `Perezosoft.Core.csproj` in its entirety:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Nine lines. Everything else lives at the solution root, where it applies to *all* projects at once — which is our next move.

3. One build policy for the whole solution

MSBuild reads a file named `Directory.Build.props` from every ancestor directory of a project. Put one at the repo root and every project inherits it — a single place for build policy, impossible for a new project to forget. This is the reference platform's actual file:

```
<Project>

  <!--
    Solution-wide build hygiene. Nullable reference types on, and warnings are
    errors so the invariants the analyzers and the compiler check can't rot into
    ignored noise. This is half of "solid by construction" — the architecture
    tests are the other half, and CI runs both.
  -->
  <PropertyGroup>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ImplicitUsings>enable</ImplicitUsings>
    <!--
      Supply-chain hardening. Restore writes a packages.lock.json next to each
      project pinning the full transitive graph; those files are committed and CI
      restores in locked mode so any drift fails the build. Paired with Central
      Package Management (Directory.Packages.props).
    -->
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>

</Project>
```

Two of these settings deserve a hard look, because both are *decisions about failure*:

`TreatWarningsAsErrors` — a warning is the compiler telling you something is probably wrong. On a codebase that tolerates warnings, the count creeps from 3 to 300, and the one that mattered drowns. Turning warnings into errors on day one costs nothing (there is no code yet!) and means the count stays at zero forever. Adopting this on an *existing* codebase is a month of cleanup — which is exactly why it must happen now, in the empty solution. Several rules you'll meet later (like nullability soundness) are only real because a violation refuses to compile.

`Nullable` — nullable reference types turn "I forgot this could be null" from a production `NullReferenceException` into a compile-time squiggle — and with the previous setting, into a build failure. The domain modeling you'll do from Part 2 onward leans on this constantly: `string?` in an entity is a *documented decision* that the value is optional, checked by the compiler, rather than tribal knowledge.

4. Central Package Management — one version table

Without CPM, every `.csproj` carries its own `<PackageReference Include="X" Version="Y">` and the same package drifts across projects (the reference repo actually hit this: two test projects referencing different versions of the test SDK). With CPM, versions live in **one** committed table, `Directory.Packages.props`, and project files reference packages *without* versions. An excerpt of the real table:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>

  <ItemGroup>
    <!-- App / API -->
    <PackageVersion Include="DotNetEnv" Version="3.2.0" />
    <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer"
Version="10.0.9" />

    <!-- Infrastructure -->
    <PackageVersion Include="MailKit" Version="4.17.0" />
    <PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.2" />

    <!-- Tests -->
    <PackageVersion Include="xunit" Version="2.9.3" />
    <PackageVersion Include="Testcontainers.PostgreSql" Version="4.12.0" />
    <!-- Consolidated: E2E referenced 17.14.0, the xUnit projects 17.14.1 → highest wins.
-->
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
  </ItemGroup>
</Project>
```

Create yours with just the packages the walking skeleton needs (the test SDK, xunit); every later lesson that says "add package X" means: add the `<PackageVersion>` row here, add a version-less `<PackageReference>` in the project that uses it. One table to review when you upgrade; one diff line when a version changes; zero possibility of the same package at two versions (that's rule R27, and it's enforced by construction).

Notice that comment about `17.14.0` vs `17.14.1` — it's a fossil of the exact drift CPM exists to prevent, found *during* the consolidation and preserved in the reference repo as a comment. Real codebases carry these scars; good ones label them.

5. Lock it — restore becomes reproducible

`RestorePackagesWithLockFile` (already in our props file) makes restore write a `packages.lock.json` beside each project: the **full transitive closure** — every package your packages depend on, recursively, with exact versions and content hashes. Commit these files. Then the magic flag:

```
dotnet restore --use-lock-file # first restore: writes the lockfiles
git add **/packages.lock.json
dotnet restore --locked-mode # from now on (and in CI): verify, don't resolve
```

In locked mode, restore doesn't *resolve* versions — it *verifies* them. If anything in the graph differs from the lockfile (a floating version moved, a transitive dependency was swapped, a hash doesn't match), restore **fails loudly** instead of silently building something new. When lesson 1.6 stands up CI, `--locked-mode` is one of its first gates: a supply-chain change becomes a red build and a reviewable diff, not an invisible drift.

The mental model: your dependency graph is now *code*. It changes by commit, with review, or not at all.

6. Run it

```
dotnet build
# 0 Warning(s), 0 Error(s) — and it must stay that way: warnings ARE errors now
dotnet restore --locked-mode
# succeeds — graph matches the committed lockfiles
```

And prove a gate actually gates (pain now, so you trust it later): edit a `packages.lock.json` by hand — change any version digit — and run `dotnet restore --locked-mode` again. It fails with `NU1004`. Revert. That failure is what a supply-chain surprise will look like: loud, early, and in your terminal instead of in production.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how do we manage dependency versions? (a) Per-project references with whatever versions were current at add-time; (b) a shared props file with version *variables*; (c) Central Package Management + committed lockfiles + locked-mode CI.

Chosen: (c) — CPM for one reviewable version table (R25), lockfiles for the transitive graph, locked restore so drift fails the build (R27). Also decided here: warnings-as-errors and nullable from the empty solution, because both are nearly free now and prohibitively expensive to retrofit.

Rejected: (a) is how version drift and silent supply-chain changes happen — it's the default, which is the problem. (b) centralizes the *numbers* but not the *graph*: transitive dependencies still float.

The trade: every dependency bump is now an explicit two-file diff (table + lock), and a new package is a two-step add. Accepted: that friction *is* the review point.

8. Checkpoint

```
git add -A && git commit -m "feat: solution skeleton — projects, inward refs, CPM, locked restore"
git tag lesson-1.1
```

You should now be able to: explain why `Core` referencing nothing is the architecture's first enforcement; add a package the CPM way; say what a lockfile pins that a version number doesn't; and articulate why warnings-as-errors is a day-one decision, not a someday cleanup.

Next: *Lesson 1.2 — First endpoint, first test* — the API says hello, and the Red→Green loop you'll live fifty times gets its first turn.

Lesson 1.2 — First endpoint, first test

> **Part 1 · The walking skeleton.** The solution compiles but does nothing. By the end of this lesson it answers HTTP — and, more importantly, a test *proves* it answers HTTP by booting the real application. This is the lesson where you live the Red→Green→Refactor loop for the first time, on the smallest possible stakes, so that when the stakes are tenant isolation or billing you already trust the rhythm.

Goal: `GET /health` returns 200, and a test in `Api.Tests` boots the actual app in-process and asserts it.

Concepts & principles: the walking skeleton; Red→Green→Refactor; in-process integration testing with `WebApplicationFactory`; testing the real composition, not a hand-assembled copy.

Maps to: ADR-C14 · repo: `src/Api/Program.cs`, `tests/Api.Tests/Infrastructure/IntegrationTestFactory.cs`, `tests/Api.Tests/Integration/HarnessSmokeTests.cs`, `src/Api/Perezosoft.Api.http`.

Prerequisites: 1.1 (solution builds, CPM in place).

---

1. Motivate — why a "hello world" deserves a whole lesson

A *walking skeleton* is the thinnest possible slice that exercises the whole path: request in, response out, test proving it. It looks trivial. It isn't, because it forces every piece of plumbing to exist and cooperate **before** any feature depends on that plumbing: the web host boots, routing routes, the test project can reach the app, CI (next lessons) can run it all. Every integration problem you flush out now is one that won't ambush you inside a complicated lesson later.

There's also a decision hiding in "a test proving it" — *what kind* of test? We could unit-test a handler function and call it a day. But the failures that hurt in web apps are rarely inside one function; they're in the *composition* — DI registrations missing, middleware ordered wrong, routes not mapped, config not read. So our default harness boots the **real** `Program` — the actual composition root, not a hand-assembled lookalike — in-process, and talks to it over real HTTP semantics. The reference platform's harness makes the reasoning explicit; from its actual doc comment:

> Boots the REAL app (`Program`) via `WebApplicationFactory<T>` … so tests exercise the full pipeline …, not hand-assembled services. … [this] de-risks routing/DI refactors by asserting routes stay stable end-to-end.

A test against a hand-wired `ServiceCollection` keeps passing while the real app is broken. A test against `Program` cannot.

2. Red — the failing test comes first

In `tests/Api.Tests`, add the package references (CPM style — versions come from the table you made in 1.1): `Microsoft.AspNetCore.Mvc.Testing`, and reference the Api project. Then write the test **before** the endpoint exists:

```
// tests/Api.Tests/HealthEndpointTests.cs
using System.Net;
using Microsoft.AspNetCore.Mvc.Testing;

namespace Perezosoft.Api.Tests;

public class HealthEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public HealthEndpointTests(WebApplicationFactory<Program> factory) => _factory =
factory;

    [Fact]
    public async Task Health_endpoint_responds_200()
    {
        var client = _factory.CreateClient(); // in-process; no port, no network

        var response = await client.GetAsync("/health");

        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    }
}
```

Run it:

```
dotnet test tests/Api.Tests
# Failed! Health_endpoint_responds_200
# Assert.Equal() Failure: Expected OK, Actual NotFound
```

Red — the app boots (WebApplicationFactory found `Program` and started the real host) but there is no `/health` endpoint, so the request falls through to a 404. That's a *properly* failing test: it fails for exactly the reason it should, and the failure message describes the missing behavior.

One wrinkle to know about before it bites: our `Program.cs` is top-level statements (no explicit class), and the class the compiler generates for it is **internal**. On current SDKs the web template arranges for the test host to reach it anyway, so the test above compiles and runs — but if you ever hit `error CS0246: The type or namespace name 'Program' could not be found`, the fix is one line in `Perezosoft.Api.csproj`, and the reference platform carries it as standing hygiene:

```
<ItemGroup>
  <InternalsVisibleTo Include="Perezosoft.Api.Tests" />
</ItemGroup>
```

`InternalsVisibleTo` is a scalpel: it grants *one named assembly* access to internals, instead of making `Program` public for the whole world. You'll see this same "smallest-possible opening" instinct again and again — it's the security posture of the entire platform in miniature. Add it now even though the build didn't force you to; the next person's SDK might.

3. Green — the minimum that passes

`src/Api/Program.cs`, in full at this stage of its life:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHealthChecks();

var app = builder.Build();

app.MapHealthChecks("/health");

app.Run();
```

```
dotnet test tests/Api.Tests
# Passed! - 1 test
```

That's the loop: a failing test that *specifies*, then the minimum code that *satisfies*. Resist the urge to add more — no controllers, no Swagger, no logging config. Every one of those will arrive in the lesson where a test demands it. (For the curious: this file grows to ~300 lines by Part 8 in the reference repo, and every block of it traces to a lesson.)

Why `AddHealthChecks()`/`MapHealthChecks` rather than a hand-rolled `app.MapGet("/health", …)`? Because this endpoint has a *career* ahead of it: in Lesson 4.5 it splits into liveness (`/health`) and readiness (`/health/ready`) with a real database probe behind a tag filter, and in Lesson 8.3 the deploy pipeline's smoke gate calls it to decide whether a release goes out. Using the framework's health-check pipeline from day one means that growth is additive. The reference repo's final form, for a preview of where this line of code is headed:

```
app.MapHealthChecks("/health", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = check =>
check.Tags.Contains("ready") });
```

4. Refactor & harden — a scratchpad and a habit

No production refactor needed yet (there's barely any production). But two small work-habits start here.

The `.http` scratchpad. Create `src/Api/Perezosoft.Api.http` — using the https port from *your* `src/Api/Properties/launchSettings.json` (`dotnet new web` assigns a random one; the reference repo happens to sit on 7160):

```
@host = https://localhost:7160

GET {{host}}/health
```

Visual Studio and VS Code (REST Client) execute these files in-editor. Every lesson that adds an endpoint adds a request here — by Part 7 it's a living, executable catalog of the whole API surface, versioned with the code. It's the cheapest API documentation that can't go stale, because you *use* it to poke the app.

Run the real thing. Tests passing in-process is necessary, not sufficient — actually boot it once:

```
dotnet run --project src/Api
# listening on https://localhost:xxxx
curl -k https://localhost:xxxx/health
# Healthy
```

The response body `Healthy` comes from the health-check middleware's default writer. In-process tests + a real boot = the two views you'll keep cross-checking all course.

5. Run it

```
dotnet build # 0 warnings, 0 errors — still, and always
dotnet test tests/Api.Tests
# 1 passed
```

6. Architecture Decision

ARCHITECTURE DECISION

The fork: what does the default test harness boot? (a) Nothing — unit-test handlers as plain functions; (b) a hand-assembled `ServiceCollection` with the pieces each test needs; (c) the real `Program` via `WebApplicationFactory`, in-process.

Chosen: (c) as the default for anything HTTP-shaped (ADR-C14's integration layer). The composition root is where web apps actually break — missing registrations, middleware order, unmapped routes — and only booting the real one tests it. In-process means no ports, no network flakes, parallel-safe.

Rejected: (a) alone leaves the wiring untested (we still unit-test pure logic — Core.Tests exists for exactly that). (b) is the seductive trap: it *looks* like an integration test but silently diverges from the real app — a test passing against a copy proves nothing about the original.

The trade: booting the real app makes some things awkward on purpose — config the app reads at startup must genuinely exist (you'll feel this in 1.4, and the reference harness has a documented seam for it: the JWT secret must be an *environment variable* because `Program` reads it before the factory's config hooks run). That awkwardness is information: it's your startup contract made visible.

7. Checkpoint

```
git add -A && git commit -m "feat: walking skeleton — health endpoint + real-app test
harness"
git tag lesson-1.2
```

You should now be able to: explain what a walking skeleton de-risks; write the failing test first and feel why the order matters; say why `WebApplicationFactory<Program>` beats a hand-assembled service collection; and explain what `InternalsVisibleTo` grants and why it's preferable to making things public.

Next: *Lesson 1.3 — Database & the test container* — Postgres joins the skeleton, and the tests get a real database that appears and disappears on demand.

Lesson 1.3 — Database & the test container

> **Part 1 · The walking skeleton.** The skeleton gets a spine: PostgreSQL via EF Core, a first migration, and — the real prize — a test fixture that gives every test run a genuine Postgres that appears in seconds and vanishes without trace. The single most consequential testing decision of the course happens here: **we do not use the EF in-memory provider. Ever.** By the end you'll know exactly why.

Goal: `AppDbContext` exists with one throwaway entity, a migration creates its schema, and a test writes to and reads from a *real* Postgres spun up by the test run itself.

Concepts & principles: EF Core migrations discipline; Testcontainers; why in-memory database fakes lie; model-derived test resets (isolation that can't be forgotten).

Maps to: ADR-C6/C7 · R11 · repo: `src/Infrastructure/Persistence/AppDbContext.cs`, `src/Api/AppDbContextFactory.cs`, `tests/Api.Tests/Infrastructure/PostgresFixture.cs`, `tests/Api.Tests/Infrastructure/PostgresTestBase.cs`, `tests/Api.Tests/MigrationsTests.cs`.

Prerequisites: 1.2 (harness runs); 0.2 (Docker running — Testcontainers needs the daemon, and you already have it for compose).

---

1. Motivate — your tests are only as honest as their database

Here's the trap nearly every .NET team falls into once. EF Core ships an in-memory provider; it's right there; tests using it need no Docker and run fast. So the test suite grows on it. Then one day a query that passed a thousand test runs fails in production, because the in-memory provider isn't a database — it's a `List<T>` wearing a trench coat. The reference platform's fixture states the consequence as a matter of record, in its actual doc comment:

> Real Postgres (not the EF in-memory provider) is required because several services branch on `Database.IsRelational()` and use relational-only constructs — `ExecuteUpdateAsync`, savepoints — that the in-memory provider silently skips.

"Silently skips" is the killer phrase. And the list is longer than that comment: the in-memory provider has no transactions worth the name, no unique constraints, no `SKIP LOCKED` (Lesson 4.1 depends on it), case-insensitive string comparisons where Postgres is case-sensitive — and, fatally for this course, its handling of **global query filters** (Lesson 2.6, the keystone of tenant isolation) diverges from a real relational provider. A tenant-isolation test that passes in-memory proves *nothing*.

The classical escape was a shared "test database" on somebody's server — slow to reset, polluted by yesterday's runs, fought over by CI agents. Testcontainers dissolves the dilemma: the test run itself starts a pristine `postgres:17` in a Docker container, uses it, and destroys it. Real engine, zero residue.

2. The context and its first entity

In `Infrastructure`, add the EF packages (CPM rows + version-less references — `Npgsql.EntityFrameworkCore.PostgreSQL`; `Microsoft.EntityFrameworkCore.Design` goes in the Api project for tooling). Then the context — humble at this stage:

```
// src/Infrastructure/Persistence/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

namespace Perezosoft.Infrastructure.Persistence;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        // Per-entity mapping lives in one IEntityTypeConfiguration<TEntity> class each,
        // discovered from this assembly — no central registry to forget.
        builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

That `ApplyConfigurationsFromAssembly` line is a pattern you met in 0.2 (health checks) and will meet a dozen more times: **discovery over registration**. Entity configurations will be *found* because they exist, not because someone remembered to list them. (In 2.6, tenant filters attach the same way — by reflection over the model — and in this lesson's fixture, the reset list will too. One idea, three appearances already.)

For a first entity we need something disposable — the real platform's first entities (User, Tenant) arrive with Part 2 and deserve their own lessons. Use a placeholder `Probe` entity with an `Id` and a `Note` — its only job is to prove the pipeline, and you'll delete it in 2.1 without ceremony. Give it an `IEntityTypeConfiguration<Probe>` so the discovery pattern gets exercised.

Now scroll your build output before moving on — there's a landmine here worth defusing in daylight. Depending on package versions, adding the EF stack can produce:

```
warning MSB3277: Found conflicts between different versions of
"Microsoft.EntityFrameworkCore" that could not be resolved.
```

...and the build still says **succeeded**. Didn't we make warnings errors in 1.1? We did — for *compiler* warnings. `MSB3277` is an **MSBuild** warning, a different pipeline that `TreatWarningsAsErrors` does not cover. Your zero-warnings bar has a blind spot, and this is the classic thing that hides in it: the Npgsql provider pulling one EF Core version while the Design/test packages pull another. The fix is to pin the shared assembly explicitly — add `Microsoft.EntityFrameworkCore.Relational` at the same version as your other EF packages to the CPM table and reference it from `Api.Tests` (the reference repo carries exactly this pin, with a comment saying why). The transferable lesson: **"the build is green" and "the build is clean" are different claims** — read the output when you add a dependency, because version-unification warnings are how runtime `MethodNotFound` surprises introduce themselves politely first.

3. Migrations — schema changes as reviewable code

The EF tooling is a separate global tool, not part of the SDK — install it once (0.2's toolchain table now has a third row):

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate --project src/Infrastructure --startup-project
src/Api
```

...and it fails, or worse, half-works, because EF's design-time tooling wants to *boot your app* to obtain a configured `DbContext` — and your app (from 1.4 onward) will refuse to boot without validated config. The reference platform closes this with a **design-time factory**, and its doc comment explains the whole story:

> Design-time factory so EF tooling ... can build the context WITHOUT booting the API host — routing through the host runs startup config validation (e.g. `Jwt:Secret`) that isn't present at design time. The connection string comes from `ConnectionStrings__DefaultConnection`, the SAME single source the running app uses ... There is no hardcoded fallback.

```
// src/Api/AppDbContextFactory.cs (abridged from the real file)
public class AppDbContextFactory : IDesignTimeDbContextFactory<AppDbContext>
{
    public AppDbContext CreateDbContext(string[] args)
    {
        try { DotNetEnv.Env.TraversePath().Load(); } catch { /* no .env present */ }

        var connectionString =
            Environment.GetEnvironmentVariable("ConnectionStrings__DefaultConnection")
            ?? throw new InvalidOperationException(
                "ConnectionStrings__DefaultConnection is not set. Add it to .env ...");

        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(connectionString).Options;
        return new AppDbContext(options);
    }
}
```

Note what it *refuses* to do: no hardcoded `Host=localhost;...` fallback. The connection string lives in exactly one place — your `.env` (ADR-001, given teeth in 0.2) — and if it's missing, the error says precisely what to fix. A fallback would "work" and thereby hide misconfiguration until it mattered.

Which means two files gain a line right now, honoring 0.2's contract that every key the system reads is documented. In `.env` (so the tooling works on your machine) and in `.env.example` (so it works on the next machine):

```
# REQUIRED — Postgres connection for the API + EF tooling. Mirror the DB_* values above
# (note the port: 5433 if you kept 0.2's collision-avoiding default).
ConnectionStrings__DefaultConnection="Host=localhost;Port=5433;Database=dev_db;Username=dev
;Password=devpassword"
```

(When 1.4's doc-sync gate arrives, an undocumented key like this would fail the build — we're just early.)

Run the migration command again; a `Migrations/` folder appears in Infrastructure. Read the generated file once, top to bottom — you'll rarely need to again, but you should know what the tool writes on your behalf: an `Up()` building your tables, a `Down()` reversing it, and a model snapshot the *next* migration diffs against. Migrations are why "the schema" is never a rumor: it's the sum of committed, reviewed, replayable scripts.

4. The fixture — a real database that can't leak between tests

Now the centerpiece. Add `Testcontainers.PostgreSql` to `Api.Tests` and build the fixture (this is the reference platform's real code, trimmed of the tenancy parts that arrive in Part 2):

```
// tests/Api.Tests/Infrastructure/PostgresFixture.cs
public sealed class PostgresFixture : IAsyncLifetime
{
    private readonly PostgreSqlContainer _container = new
PostgreSqlBuilder("postgres:17").Build();

    public string ConnectionString => _container.GetConnectionString();

    public async Task InitializeAsync()
    {
        await _container.StartAsync();
        await using var db = CreateContext();
        // Build the schema from the EF model (fast; no migration history needed for
tests).
        await db.Database.EnsureCreatedAsync();
    }

    public Task DisposeAsync() => _container.DisposeAsync().AsTask();

    public AppDbContext CreateContext()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(ConnectionString).Options;
        return new AppDbContext(options);
    }

    /// <summary>Truncates every table so each test starts from a clean slate. The table
    /// list is DERIVED from the EF model, so a new entity is reset automatically — no
    /// hand-maintained list to forget.</summary>
    public async Task ResetAsync()
    {
        await using var db = CreateContext();
        var tables = db.Model.GetEntityTypes()
            .Select(e => e.GetTableName())
            .Where(name => name is not null)
            .Distinct()
            .Select(name => $"\"{name}\"");
        var sql = "TRUNCATE TABLE " + string.Join(", ", tables) + " RESTART IDENTITY
CASCADE;";
        await db.Database.ExecuteSqlRawAsync(sql);
    }
}

[CollectionDefinition(PostgresCollection.Name)]
public sealed class PostgresCollection : ICollectionFixture<PostgresFixture>
{
    public const string Name = "postgres";
}
```

Three design decisions in forty lines, each load-bearing:

One container per run, not per test. Container startup costs seconds; a suite of hundreds of tests can't pay it each time. The xUnit *collection fixture* shares one container across every test class marked `[Collection(PostgresCollection.Name)]`, and xUnit runs a collection's members serially — so sharing is safe by construction.

Reset between tests, and make the reset un-forgettable. Shared container means shared state, so every test must start clean. The rookie version is a hand-maintained list of tables to truncate — which goes stale the day someone adds an entity and forgets the list (this exact bug is why rule R11 exists; the reference repo found it in an audit). The fix is the discovery pattern again: **derive the table list from `db.Model.GetEntityTypes()`**. New entity → automatically in the reset. And rather than trusting each test to call `ResetAsync()`, a tiny base class makes isolation structural:

```
// tests/Api.Tests/Infrastructure/PostgresTestBase.cs — real file, in full
public abstract class PostgresTestBase(PostgresFixture fixture) : IAsyncLifetime
{
    protected PostgresFixture Fixture { get; } = fixture;

    public Task InitializeAsync() => Fixture.ResetAsync(); // before EVERY test
    public Task DisposeAsync() => Task.CompletedTask;
}
```

Inherit it and there is no reset call to forget. Same move as the tenant filter to come: correct by default, not correct by discipline.

`EnsureCreatedAsync` here, migrations elsewhere. The fixture builds schema straight from the model — fast, no history table. But that means the *migrations themselves* could rot untested. So one more test exists solely to apply every migration, in order, against a scratch database (`MigrationsTests` in the reference repo — and the integration harness from 1.2 will boot the app with its real startup-migration path in Part 2). Schema-from-model for speed; migrations proven separately; nothing unverified.

5. Red → Green — prove the loop against real Postgres

```
// tests/Api.Tests/ProbePersistenceTests.cs
[Collection(PostgresCollection.Name)]
public class ProbePersistenceTests(PostgresFixture fixture) : PostgresTestBase(fixture)
{
    [Fact]
    public async Task Probe_roundtrips_through_real_postgres()
    {
        await using (var db = Fixture.CreateContext())
        {
            db.Add(new Probe { Note = "hello from a container" });
            await db.SaveChangesAsync();
        }
        await using (var db2 = Fixture.CreateContext()) // fresh context: no cache tricks
        {
            var probe = await db2.Set<Probe>().SingleAsync();
            Assert.Equal("hello from a container", probe.Note);
        }
    }
}
```

```
dotnet test tests/Api.Tests
# Passed! - 2 tests (health + probe) ... first run pulls postgres:17, be patient
```

Watch `docker ps` during the run: a postgres container blinks into existence, then is gone. No cleanup script, nothing to remember, nothing left behind.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: what database do tests run against? (a) EF in-memory provider; (b) SQLite in-memory ("at least it's relational"); (c) a shared long-lived test server; (d) Testcontainers — throwaway real Postgres per run.

Chosen: (d), with schema-from-model for speed and a separate migrations-apply test. Tests must exercise the engine production runs, because this platform *leans* on Postgres-specific behavior: global query filters as the tenancy wall, `SKIP LOCKED` job claiming, savepoints, and eventually row-level security. (ADR-C7/C13; R11 for the derived reset.)

Rejected: (a) silently skips the exact constructs we depend on — a green suite proving nothing. (b) is a different engine with different SQL, different concurrency, no RLS: it converts "provider lies" into "dialect lies". (c) reintroduces shared mutable state across runs and machines — the thing test isolation exists to kill.

The trade: Docker becomes a hard requirement for running the suite, and the first test run pays an image pull. Accepted consciously in 0.2 — the same daemon already runs your dev infrastructure, and CI (1.6) has it natively.

7. Checkpoint

```
git add -A && git commit -m "feat: EF Core + Postgres — first migration, Testcontainers fixture, model-derived reset"
git tag lesson-1.3
```

You should now be able to: list three concrete ways the in-memory provider lies; explain the container-per-run / reset-per-test economy and why the reset list is derived, not written; say what a design-time factory is for and why it must not have a connection fallback; and describe how migrations stay verified even though the fixture doesn't use them.

Next: *Lesson 1.4 — Configuration & the options pattern* — the app learns to read config, validate it at startup, and fail loudly at boot rather than mysteriously at 3 a.m.

Lesson 1.4 — Configuration & the options pattern

> **Part 1 · The walking skeleton.** The app can answer HTTP and talk to Postgres; now it learns to be *configured* — and, more to the point, to refuse to start when it isn't. Configuration is where "works on my machine" goes to hide: a missing key that defaults to something plausible, a secret pasted into a JSON file, a setting the code reads that nobody documented. This lesson closes all three holes, and closes them with machinery, not vigilance.

Goal: the API loads `.env` in dev, exposes typed, validated settings objects, fails at *startup* (not first-use) on missing/invalid config, and a test fails the build when code reads a config key that isn't documented.

Concepts & principles: the config hierarchy; secrets never in the repo (ADR-001 mechanized); typed settings with construction-time validation; fail-fast startup; the doc-sync gate.

Maps to: ADR-001 · R20–R22 · repo: `src/Api/Configuration/SettingsProvider.cs`, `SettingsRegistration.cs`, `src/Api/Program.cs` (the DotNetEnv line), `tests/Api.Tests/DocAndConfigSyncTests.cs`, `.env.example`, `appsettings.json`.

Prerequisites: 1.3; the `.env` contract from 0.2.

---

1. Motivate — the 3 a.m. config bug

Picture the failure mode we're designing against. A service reads `config["Jwt:Secret"]` deep inside a request handler. On the machine where it's missing, nothing happens at startup — the app boots, health checks pass, the deploy is declared good. Hours later the first user tries to sign in and gets a 500, and the stack trace points at token-signing code, three layers away from the actual problem: a key nobody set. Now multiply by every config value in the system.

The root defect isn't the missing key — machines will always occasionally be misconfigured. The defect is **when** and **how** the system tells you. Config errors should surface at *boot*, in a message that names the key and says where to set it. That transformation — from runtime mystery to startup message — is what this lesson builds.

And beneath it, a second rule from Lesson 0.1's ADR-001: secrets never live in committed files. Which raises the practical question: where *does* the app get them, and how do committed files and secret files cooperate?

2. The config hierarchy — one mental model

ASP.NET Core builds `IConfiguration` from layered sources, later layers overriding earlier ones. Our arrangement (dev):

```
appsettings.json committed — non-secret defaults, shape documentation
appsettings.Development.json committed — dev-only tweaks (verbose logging, etc.)
environment variables NOT committed — secrets & machine-specifics …
        ▲
        └── in dev, populated from .env by DotNetEnv at startup
```

The elegant part is the last line. Production sets real environment variables; dev keeps them in the gitignored `.env` from 0.2, and a single line at the very top of `Program.cs` — before anything reads config — loads them. From the real file:

```
// Local dev: load secrets/config from the repo-root .env (the single local source of
// truth — see docs/DECISIONS.md). TraversePath walks up to find it regardless of the
// working dir; the try/catch makes it a no-op when there's no .env (e.g. production,
// which uses real environment variables).
try { DotNetEnv.Env.TraversePath().Load(); } catch { /* no .env present */ }

var builder = WebApplication.CreateBuilder(args);
```

One mechanism, two environments: to the app, dev and prod look identical — env vars either way. Nested keys use the `Section__Sub` double-underscore convention (`Jwt__Secret` in `.env` becomes `Jwt:Secret` to `IConfiguration`), which is the standard env-var transport for hierarchical config.

`appsettings.json` keeps the **non-secret** shape: section names, defaults worth committing, values a reviewer should see. It also serves as *documentation of what exists* — which the gate in §5 exploits.

3. Typed settings — validation at construction

Reading `config["Jwt:Secret"]` at call sites scatters string literals through the code, defers errors to first use, and re-parses on every read. The platform's pattern: one small class per config section, reading and validating **in its constructor**, registered as a singleton. The real `JwtSettings`:

```
// src/Api/Configuration/SettingsProvider.cs
public class JwtSettings : IJwtSettings
{
    public string SecretKey { get; }
    public string Issuer { get; }
    public int ExpiryMinutes { get; }

    public JwtSettings(IConfiguration config)
    {
        SecretKey = config["Jwt:Secret"]
            ?? throw new InvalidOperationException(
                "Jwt:Secret not configured (set Jwt__Secret in .env for dev)");
        Issuer = config["Jwt:Issuer"] ?? "Perezosoft";
        ExpiryMinutes = config.GetValue("Jwt:ExpiryMinutes", 60);

        const int MinSecretKeyLength = 32;
        if (SecretKey.Length < MinSecretKeyLength)
            throw new InvalidOperationException(
                $"Jwt:Secret must be at least {MinSecretKeyLength} characters");
    }
}
```

Read the error messages as UX, because they are: each names the key **and the fix** ("set `Jwt__Secret` in `.env` for dev"). Whoever hits this — possibly you, in six months, on a new laptop — gets handed the solution, not a scavenger hunt. And notice the *semantic* validation: not just "present" but "at least 32 characters," because an HMAC key shorter than that is a security bug that would otherwise hide until token validation mysteriously failed.

Registration happens once, in one place, and the singletons are constructed **eagerly** so a bad value kills the boot:

```
// src/Api/Configuration/SettingsRegistration.cs (abridged — real file)
public static IJwtSettings AddAppSettings(this IServiceCollection services, IConfiguration
config)
{
    // Build JwtSettings once; the same instance backs both the DI singleton and the
    // JWT-bearer TokenValidationParameters — a single source of truth.
    var jwtSettings = new JwtSettings(config);

    services.AddSingleton<IJwtSettings>(jwtSettings);
    services.AddSingleton<IRefreshTokenSettings>(new RefreshTokenSettings(config));
    services.AddSingleton<IApplicationSettings>(new ApplicationSettings(config));
    // ...
    return jwtSettings;
}
```

Two subtleties worth pausing on. The settings are constructed with `new`, *right here*, not lazily by the container — so startup **is** the validation pass; a missing key can't lurk until first resolve. And `JwtSettings` is built into a local and *returned*, because `Program.cs` needs the same values to configure JWT-bearer validation — one instance, two consumers, zero chance of drift between "the settings DI hands out" and "the settings auth actually validates with." (This is the platform's blessed shape per rule R22; .NET's `IOptions` + `ValidateOnStart` is a fine alternative — the *decision* is eager validation and one pattern everywhere, not which library performs it.)

Consumers then depend on `IJwtSettings` — an interface, so tests hand in a stub without touching `IConfiguration` at all. Config-reading is quarantined to these constructors the way MailKit is quarantined to `Infrastructure/Email`: one place, one pattern.

And your 1.2 test just broke — good. `WebApplicationFactory` boots the *real* `Program`, which now demands `Jwt:Secret` at startup; the health test dies with the very `InvalidOperationException` you wrote. This is fail-fast config working exactly as designed — the test boots the app, the app validates its config, no exceptions for test mode. The seam is the one the reference harness documents: `Program` reads config at `CreateBuilder` time, *before* the factory's config hooks run, so the value must exist as an **environment variable** that early. A static constructor on the test class is the cleanest place:

```
public class HealthEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    static HealthEndpointTests() =>
        Environment.SetEnvironmentVariable("Jwt__Secret",
            "integration-test-jwt-secret-key-0123456789"); // ≥32 chars; value irrelevant
    // ...
}
```

The dummy only needs to *satisfy the validation* (length), because nothing in this test signs or verifies a token. When the integration harness grows up in Part 2, this seam moves into its

constructor — same idea, one shared home.

4. Defaults are decisions

Look again: `Issuer` defaults to `"Perezosoft"`, `ExpiryMinutes` to `60` — but `SecretKey` throws. Why the asymmetry? Because **a default is a decision that absence is safe**. A missing issuer name has a sensible universal answer. A missing *secret* does not — any default would be catastrophic, so absence must be fatal. When you write a settings class, sort every value into one of these bins deliberately: safe default, or refuse to boot. (Part 7 adds a third bin for optional *features*: absent config means the feature is **off** — R21's config-gated default-off. Same principle: absence maps to the safe state, never the surprising one.)

5. The gate — config that can't go undocumented

Now the structural guarantee. Nothing so far stops a developer from reading `config["Fancy:NewKnob"]` in code and telling no one — until someone deploys without it. The reference platform closes this with a test that cross-references **code against documentation**, from the real `DocAndConfigSyncTests`:

```
[Fact]
public void ConfigKeys_ReadInCode_AreDocumented()
{
    // R20: every Section:Key literal read via IConfiguration is documented — either in
    // an appsettings*.json (as a nested path) or in .env.example (as Section__Key).
    // Catches config drift where code reads a key nobody declared.
    var documented = DocumentedConfigPaths(); // parsed from appsettings*.json +
.env.example

    var readKeys = new HashSet<string>(StringComparer.Ordinal);
    var access = new Regex(
        @"(?:GetSection|GetValue<[^>]*>|GetValue|configuration|config|Configuration)\s*[\(\
[]\s*""([A-Za-z][A-Za-z0-9]*(?::[A-Za-z0-9]+)+)""");
    foreach (var f in SourceFiles(Path.Combine(RepoRoot(), "src")))
        foreach (Match m in access.Matches(File.ReadAllText(f)))
            readKeys.Add(m.Groups[1].Value);

    var undocumented = readKeys.Where(k => !documented.Any(d =>
            d.Equals(k) || d.StartsWith(k + ":") || k.StartsWith(d + ":")))
        .OrderBy(k => k).ToList();

    Assert.True(undocumented.Count == 0,
        $"Config keys read in code but not in appsettings*.json or .env.example:
{string.Join(", ", undocumented)}");
}
```

It's a source scan — grep with a build verdict. Read a key in code without adding it to `.env.example` or an `appsettings*.json`, and the suite goes red with the key's name in the failure message. (One adaptation when you write `RepoRoot()`: anchor the upward directory walk on a file your repo actually has at its root — `global.json` works perfectly and 0.2 guaranteed it exists.) The documentation *cannot* drift from the code, because drift fails CI. This is the third face of one idea you now hold: discovery over registration (1.3), isolation by base class (1.3), documentation by gate (here). None of them ask a human to remember anything.

Add the corresponding entries to `.env.example` as you go — from 0.2 it's the contract: every key, a comment saying what it does, required-or-default status. `Jwt__Secret=` goes in now

(empty value, comment "REQUIRED — ≥32 chars").

6. Run it — watch it fail correctly

The best test of fail-fast config is failing it on purpose:

```
# comment out Jwt__Secret in your .env, then:
dotnet run --project src/Api
# Unhandled exception: System.InvalidOperationException:
# Jwt:Secret not configured (set Jwt__Secret in .env for dev)
```

Boot refused, key named, fix stated. Restore the key; boots clean. Then run the suite — including your new doc-sync test — and prove the gate: read a fake key (`config["Nope:Missing"]`) anywhere in `src/`, watch the test name it, delete the line.

```
dotnet test tests/Api.Tests # all green, including ConfigKeys_ReadInCode_AreDocumented
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does configuration reach the code? (a) `config["key"]` at call sites; (b) .NET's options pattern (`IOptions<T>` + `BindConfiguration` + `ValidateOnStart`); (c) hand-rolled typed settings classes validating in their constructors, registered eagerly as singletons.

Chosen: (c) as this platform's single blessed pattern (R22), with ADR-001's `.env` as the dev secret source and a doc-sync test (R20) making documentation mandatory. Constructor validation gives fail-at-boot with zero framework ceremony, interfaces make consumers trivially testable, and *one pattern everywhere* means a new reader learns it once.

Rejected: (a) scatters literals, defers failure to first use, and is exactly what the R20 gate exists to catch. (b) is genuinely fine — `ValidateOnStart` achieves the same fail-fast — but brings `IOptions<T>` indirection at every consumer and *two* ways to do one thing once any hand-rolled class exists. Uniformity won. The pattern choice matters less than the properties: eager, validated, typed, documented.

The trade: hand-rolled means we own the validation code, and each new section is a small class rather than one `Bind` call. Accepted for a platform this size; revisit if settings classes multiply into the dozens.

8. Checkpoint

```
git add -A && git commit -m "feat: typed validated config — .env loading, fail-fast
settings, doc-sync gate"
git tag lesson-1.4
```

You should now be able to: draw the config layering and say where secrets live in dev vs prod; explain why validation belongs in the constructor and registration must be eager; sort a config value into "safe default" vs "refuse to boot" and defend the sort; and describe how the doc-sync gate turns documentation from a request into an invariant.

Next: *Lesson 1.5 — The error envelope* — deciding, once, what every error in the system looks like from the outside.

Lesson 1.5 — The error envelope

> **Part 1 · The walking skeleton.** The shortest lesson in the course, on purpose — the artifact is five lines. But it's a five-line *treaty*: a decision about what every error in the system looks like from the outside, made once, before there are any errors to regret. Small decisions made early are cheap; the same decisions retrofitted across forty endpoints are a migration project.

Goal: one shared error shape for the whole API; a written rule that exception internals never cross the boundary; and clarity on what an error *code* is for versus an error *message*.

Concepts & principles: the error envelope; machine-readable vs human-readable error channels; information leakage through error text; deciding API contracts before the API grows.

Maps to: R16, R18 · repo: `src/Api/Services/ErrorResponse.cs`, its uses across every controller.

Prerequisites: 1.2 (an API exists to have errors).

---

1. Motivate — the anatomy of a leaked stack trace

Two failure modes, one root cause.

Failure one: the inconsistent zoo. Without a decision, each endpoint invents its own error shape: `{ "error": "..." }` here, `{ "message": "..." }` there, `{ "errors": [ ... ] }` from whoever wrote validation, a bare string from the intern's endpoint, HTML from the framework when nobody handled it at all. Every client — your own Blazor app first of all — grows a thicket of "well, if the body has *this* shape…" parsing. The API's error surface becomes something you *reverse-engineer* rather than read.

Failure two: the confession. Somewhere, a handler does the natural thing:

```
catch (Exception ex)
{
    return StatusCode(500, new { error = ex.Message }); // ← the natural thing. Don't.
}
```

Now consider what `ex.Message` can contain, on a bad day: `"28P01: password authentication failed for user \"app_rt\""` — a Postgres error naming an internal role. `"Connection refused (10.0.3.17:5432)"` — internal topology. `"The token 'eyJhbGciOi…' is malformed"` — a credential fragment. An attacker probing your API doesn't need a vulnerability report; your exceptions *are* one, streamed on demand. This is why rule R16 exists and is checked in review: **client-facing error bodies never contain raw exception text; detail stays in server logs**, where you have access control and the attacker doesn't.

Both failures have the same fix: decide the shape once, make the safe thing the easy thing.

2. Green — the envelope

The reference platform's entire `ErrorResponse.cs`:

```
namespace Perezosoft.Api.Services;

/// <summary>The standard API error envelope. Serializes to
/// <c>{ "error": ..., "message": ... }</c> (camelCase) — the shape every error
/// response uses.</summary>
public record ErrorResponse(string Error, string Message);
```

That's the artifact. Its power is entirely in the *convention it anchors*. Usage, from the real `AuthController`:

```
return Unauthorized(new ErrorResponse("no_refresh_token", "Refresh token not found"));
return Unauthorized(new ErrorResponse("invalid_refresh_token", "Refresh token is invalid or
expired"));
return StatusCode(500, new ErrorResponse("refresh_failed", "Failed to refresh token"));
```

Two fields, two audiences, and the discipline is in keeping them apart:

`error` is for machines. A stable, snake_case code — `invalid_refresh_token`, `seat_limit_reached` — that clients branch on. It's *API contract*: renaming one is a breaking change, exactly like renaming a JSON property. When the Blazor client (3.4) decides whether to bounce a user to login or show "try again," it switches on this field, never on prose.

`message` is for humans. A safe, self-contained sentence for logs and debugging. It describes the *category* of failure in words you chose — never words an exception chose. Note what `"Failed to refresh token"` doesn't say: no exception type, no stack frame, no connection string. The interesting detail went to the server log at the `catch` site, correlated by the request's trace ID (which Lesson 4.5 makes automatic).

Why not ProblemDetails (RFC 7807), the standards-blessed answer? It's a reasonable fork — see the decision box. The short version: the envelope predates it here, is smaller, and the *rule* (one shape, no leaks) matters far more than which shape.

3. Harden — a shape is only a shape if it's the only one

A convention with exceptions is a suggestion. Two enforcement layers make this one hold:

Status codes carry the first bit of meaning. The envelope rides on top of correct HTTP semantics, and this platform is unusually disciplined about three of them — you'll build each: **401** "we don't know who you are" (2.1), **403** "we know you, your *role* may not do this" (6.1, `RequirePermission`), **402** "we know you, your *plan* doesn't include this" (5.1, `RequireEntitlement`). Three different problems, three different client reactions (sign in / hide the button / show the upgrade page), distinguishable before reading a byte of body. The envelope then says *which* permission, *which* limit.

A scan for defectors. Rule R18: all error responses use the shared helper — no anonymous `{ error = ... }` objects. It's enforced the same way as 1.4's config gate: a source scan that fails when an anonymous error shape appears in a controller. You now have three of these grep-with-a-verdict tests (MailKit containment, config sync, error shape) — cheap, blunt, effective. When you write your own (3.3 formalizes the habit into the architecture-test project), keep the pattern: scan for the *forbidden* form, fail with the file name and the fix.

One honest caveat: model-binding failures (malformed JSON, missing required fields) produce ASP.NET's built-in `ValidationProblemDetails` before your code runs. The platform

leaves that be — framework-generated 400s are consistent *with each other*, and bending the framework's validation pipeline into the envelope isn't worth the ceremony. The envelope rule governs *your* error paths. Record the exception to the rule as part of the rule; an undocumented exception is how conventions rot.

4. Run it

```
curl -k https://localhost:7160/api/nope
# 404 — and once real endpoints exist, error bodies all read:
# { "error": "not_found", "message": "..." }
```

The real payoff is invisible today. It arrives every time this course adds an endpoint and there is exactly one way an error can look — and it arrives hardest in 3.4, when the client's error handling is a single small function instead of a parser museum.

5. Architecture Decision

ARCHITECTURE DECISION

The fork: what do API errors look like? (a) Whatever each endpoint returns; (b) RFC 7807 ProblemDetails everywhere; (c) a minimal shared envelope (`{ error, message }`) + correct status codes, with framework validation left native.

Chosen: (c). The binding decisions are *one shape* and *no exception text* (R16, R18); a two-field record delivers both with zero ceremony, and the machine-readable `error` code gives clients a stable contract the status code alone can't.

Rejected: (a) isn't a decision, it's an absence of one — and it's what naturally happens if this lesson doesn't exist. (b) is genuinely respectable: standard, extensible, tooling-friendly. It lost on weight — `type` URIs and extension members nobody would populate — and on the platform's preference for the smallest thing that holds the rule. Migrating later is mechanical precisely because every error already flows through one type.

The trade: a custom shape means API consumers read *your* docs, not an RFC. Accepted for a platform whose first-party clients dominate; the public API surface (7.3) documents the envelope in its OpenAPI schema.

6. Checkpoint

```
git add -A && git commit -m "feat: shared error envelope — one shape, no exception leakage"
git tag lesson-1.5
```

You should now be able to: name the two audiences of an error and which field serves each; explain why `ex.Message` in a response body is a security defect, not a style issue; distinguish 401/402/403 and say who reacts to each; and defend deciding contract shapes *before* the surface grows.

Next: *Lesson 1.6 — CI from commit one* — everything this part built (locked restore, warnings-as-errors, the gates) starts running on every push, forever.

Lesson 1.6 — CI from commit one

> **Part 1 · The walking skeleton.** The final lesson of the skeleton, and the one that makes everything before it *permanent*. So far, the gates you've built — locked restore, warnings-as-errors, the doc-sync scan — run when you remember to run them. This lesson puts them on a machine that never forgets: a pipeline that executes on every push, on a clean runner, with no local state to flatter you. The pipeline is born small and — like the architecture-test suite it partners with — **grows a job per part** for the rest of the course.

Goal: a GitHub Actions workflow that restores in locked mode, builds with warnings-as-errors, runs the tests (Testcontainers included), scans for secrets and copyleft licenses — red on any violation, on every push and pull request.

Concepts & principles: CI as a growing gate; clean-room verification ("if it only works on your machine, it doesn't work"); supply-chain gates in the pipeline; the PR checklist as process.

Maps to: R26, R27 (+ every earlier rule, now enforced remotely) · repo: `.github/workflows/ci.yml` (jobs `build-test`, `secret-scan`, `license-scan`), `.github/forbidden-licenses.json`, `.github/pull_request_template.md`.

Prerequisites: 1.1–1.5 (there must be something worth gating); a GitHub repo to push to.

---

1. Motivate — a gate you add at the end is a gate you never designed for

Here's what happens to teams that "add CI later." Later arrives; someone writes the workflow; it fails — of course it fails, nothing was ever built on a clean machine. The tests assume a database that isn't there, the build has 87 warnings that were always going to be errors *someday*, a connection string is hardcoded somewhere, and one package resolves differently than it did in March. Fixing all of it is now a project, so the workflow gets watered down — warnings allowed, the flaky tests skipped — and the team learns that CI is a formality. The gate exists, but the gate gates nothing.

Now run the tape the other way. The pipeline exists from commit one, when the entire system is a health endpoint and a probe entity. It's green in five minutes of work, because there's nothing to be red *about*. From then on, every lesson that adds a capability adds its check while the diff is small and the author is present. The pipeline never has a catching-up crisis, because it never fell behind. That's the whole argument, and it's the same argument as warnings-as-errors in 1.1: **adopt the standard when meeting it is free.**

There's a second, subtler payoff. Your machine has your `.env`, your Docker images, your half-remembered global tools. A green build there is a claim contaminated by local state. CI is a *clean room*: fresh runner, no `.env`, nothing but the repo and the workflow. When it's green, the claim "anyone can build this from a checkout" has actually been tested — which is, notice, the same claim Lesson 0.2 made about dev machines. CI is 0.2's reproducibility promise, verified continuously.

2. The workflow — build-test, from the real file

Create `.github/workflows/ci.yml`. This is the reference platform's actual first job, and every step is a decision you already made — now enforced remotely:

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-dotnet@v5
        with:
          dotnet-version: "10.0.x"

      # Supply-chain: restore in LOCKED mode against the committed packages.lock.json
      # files. --locked-mode fails if the resolved graph differs from the lockfile, so a
      # dependency change can't slip in without updating the (reviewed) lockfile.
      - name: Restore (locked mode)
        run: |
          dotnet restore src/Api/Perezosoft.Api.csproj --locked-mode
          dotnet restore tests/Core.Tests/Perezosoft.Core.Tests.csproj --locked-mode
          dotnet restore tests/Api.Tests/Perezosoft.Api.Tests.csproj --locked-mode

      # Warnings-as-errors comes from Directory.Build.props — the same build policy
      # locally and here, one definition. --no-restore: reuse the locked restore above
      # (don't let the build re-restore loosely).
      - name: Build (warnings-as-errors)
        run: dotnet build src/Api/Perezosoft.Api.csproj -c Release --no-restore

      # Api.Tests spins up Postgres via Testcontainers — Docker is available on
      # ubuntu-latest, so the same tests run here as locally, unchanged.
      - name: Test (Core + Api)
        run: |
          dotnet test tests/Core.Tests/Perezosoft.Core.Tests.csproj -c Release --no-restore
          dotnet test tests/Api.Tests/Perezosoft.Api.Tests.csproj -c Release --no-restore

      # Fail the build if the EF model has drifted from the latest migration/snapshot.
      - name: No pending EF model changes
        env:
          ConnectionStrings__DefaultConnection:
"Host=localhost;Port=5432;Database=ef_designtime;Username=ci;Password=ci"
        run: |
          dotnet tool install --global dotnet-ef
          dotnet ef migrations has-pending-model-changes \
            --project src/Infrastructure/Perezosoft.Infrastructure.csproj \
            --startup-project src/Api/Perezosoft.Api.csproj
```

Walk the details, because each is a small lesson:

- **`--locked-mode` then `--no-restore`.** The restore step verifies the graph against

the lockfiles (1.1's gate, now unskippable); the build and test steps then *reuse* that restore. Without `--no-restore`, `dotnet build` would quietly re-restore in normal mode — and your

carefully locked graph would be decoration.

- **Testcontainers just works.** The GitHub runner has Docker, so the suite you built in

1.3 — real Postgres and all — runs identically here. This is the moment the Testcontainers decision pays its second dividend: no "CI database" to provision, no service-container YAML to keep in sync with the fixture.

- **The EF drift check** closes a gap you couldn't feel locally: change an entity,

forget `dotnet ef migrations add`, and everything *builds* — the model and the migrations have simply parted ways, to be discovered at the next deploy. `has-pending-model-changes` compares model to snapshot and fails loudly. Note the placeholder connection string: the check opens no DB connection, but the design-time factory (1.3) *requires the key to exist* — no fallback, remember. CI documents that contract by satisfying it explicitly.

3. Two more jobs — the supply-chain gates go remote

Secret scan. 0.2 installed gitleaks as a local backstop; policy is when it runs whether you remember or not. From the real workflow:

```
  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
        with:
          fetch-depth: 0 # full history so the scan covers every commit, not just the tip

      - name: gitleaks
        env:
          GITLEAKS_VERSION: "8.21.2"
        run: |
          curl -sSfL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VE
RSION}/gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz" \
            | tar -xz gitleaks
          ./gitleaks detect --no-banner --redact --config .gitleaks.toml
```

Two deliberate choices worth stealing: `fetch-depth: 0` scans **all history**, because a secret committed and then "deleted" is still a leaked secret — git remembers; and `--redact`, because a scanner that prints the secrets it finds into a CI log has just created a second leak. (It runs the pinned OSS binary directly rather than a marketplace action — one less third party in the trust chain of the job whose *purpose* is trust.)

License scan. Rule R26: no copyleft in the dependency graph. Not because copyleft is bad — because for *this* platform (a commercial SaaS template) a GPL dependency imposes obligations the product can't meet, and a transitive one arrives silently with some innocent-looking package. The job inventories every license, direct **and transitive**, and fails on a prohibited list (`.github/forbidden-licenses.json`: GPL/AGPL/LGPL/SSPL):

```
  license-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-dotnet@v5
        with:
          dotnet-version: "10.0.x"
      - name: dotnet-project-licenses (fail on copyleft)
        run: |
          dotnet restore src/Api/Perezosoft.Api.csproj
          dotnet tool install --global dotnet-project-licenses
          export PATH="$PATH:$HOME/.dotnet/tools"
          dotnet-project-licenses --input src/Api/Perezosoft.Api.csproj \
            --include-transitive --use-project-assets-json \
            --forbidden-license-types .github/forbidden-licenses.json
```

Note the shape: a **prohibited-list, not an allow-list** — MIT/Apache/BSD and friends never trip a false positive, so the gate stays quiet until it has something real to say. A gate that cries wolf gets disabled; this one is designed to be believed. It's also a *separate job*, deliberately: a license failure and a build failure are different conversations, and the red X should say which one you're having.

4. The human gate — the PR template

One more file, `.github/pull_request_template.md`, auto-loaded by GitHub into every pull request: the platform's checklist — tests written first, tenant-scoping verified, docs updated, no direct UI→DB access. It gates nothing mechanically; it's the *review script* — the conversation every change must survive, written down. The division of labor is worth stating: **machines check what machines can check** (this lesson, the arch tests), **the checklist scripts what humans must check** (rules like R16's "no exception text" — review-enforced until a scan exists). Process is a gate too; it just runs on people.

5. Run it — push and watch

```
git add .github && git commit -m "ci: build-test + secret-scan + license-scan on every
push"
git push -u origin main
# → GitHub → Actions: three jobs, all green
```

Then prove the gates gate (one minute, permanent trust): bump any package version in `Directory.Packages.props` *without* updating the lockfile, push a branch, and watch `Restore (locked mode)` fail with `NU1004`. Revert. That red X is 1.1's promise, kept by a machine that doesn't know you and doesn't take your word for anything.

Where this file goes from here: e2e journeys join in 3.6; the Docker image build in 8.2; deploy-to-staging with a post-deploy smoke, then gated production, in 8.3; native platform legs in the appendix. In the reference repo this file ends at ~800 lines and fourteen jobs — every one of which you'll have written in the lesson that gave it a reason to exist.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: when does CI arrive, and how much does it check? (a) Later, once there's "something real to test"; (b) now, minimal — build only, expand someday; (c) now, with every gate the codebase already claims to honor: locked restore, warnings-as-errors, tests, EF drift, secrets, licenses.

Chosen: (c). A claimed-but-unchecked standard decays silently; every gate in this job list is one the previous five lessons *already committed to*, so enforcing it remotely costs minutes now and prevents the catch-up crisis later. Growth by accretion — a job per capability, added with the capability — keeps every addition small and owned.

Rejected: (a) is how teams end up with the watered-down formality pipeline — descoped at exactly the moment it's needed most. (b) defers the cheap-now checks to when they're expensive, which is (a) with better intentions.

The trade: every push now takes a few minutes to go green, and a red X blocks merging even when "it's fine, it's just the license tool." That friction is the product. Accepted — and never bypassed with a skip flag; a gate you can wave through is a gate in name only.

7. Checkpoint

```
git tag lesson-1.6
```

Part 1 complete. You have a walking skeleton with a spine, a nervous system, and an immune system: an API answering HTTP, real-database tests, validated config, one error shape — and a clean-room pipeline that re-verifies all of it on every push, forever.

You should now be able to: argue for CI-from-day-one against "we'll add it when there's something to test"; explain `--locked-mode` + `--no-restore` as a pair; say why the secret scan reads full history and redacts; and describe the prohibited-list design and why quiet gates are trustworthy gates.

Next: *Part 2 — Identity & tenancy.* The skeleton gets people: users, tokens, and the tenant model everything else stands on.

Part 2 — Identity & tenancy

Lesson 2.5 — Tenancy & the global query filter

> **Part 2 · Identity & tenancy.** This is the keystone of the whole platform. Everything you build after this — notes, billing, files, webhooks — leans on the guarantee we install here: *a query cannot accidentally read another tenant's data.* We're going to earn that guarantee the hard way: build the leak first, prove it with a test, then close it structurally.

Goal: an `ITenantScoped` entity is filtered to the current tenant automatically, so a handler that "forgets" to scope its query still cannot read another tenant's rows.

Concepts & principles: multi-tenancy (tenant ≠ user); the difference between *filter-by-convention* and *structural* isolation; EF Core global query filters; fail-closed defaults.

Maps to: ADR-003 (tenancy via `TenantMembership` + global filter) · Golden Rule 1 · Foundation Rule R2 · repo: `src/Infrastructure/Persistence/AppDbContext.cs`, `src/Core/Entities/ITenantScoped.cs`, `src/Core/Abstractions/ICurrentTenant.cs`.

Prerequisites: 2.1 (users + JWT) and 2.2 (the `tenant_id` claim rides on the access token). You have an `AppDbContext`, a Postgres Testcontainer fixture (1.3), and a way to sign in as a user belonging to a tenant.

---

1. Motivate — build the leak, then look at it

Every SaaS bug that ends up in the news is some version of "*customer A saw customer B's data.*" The naive defense is discipline: "always add `.Where(x => x.TenantId == me)` to every query." That fails the first time a tired developer forgets — and you cannot grep your way to confidence across a growing codebase.

Let's feel it. Suppose we have a `Widget` entity and a handler that lists widgets:

```
// src/Core/Entities/Widget.cs — first attempt, NOT yet tenant-aware
public class Widget
{
    public Guid Id { get; set; }
    public Guid TenantId { get; set; } // we store it…
    public string Name { get; set; } = "";
}

// the "forgot to scope" query a real handler might contain
public Task<List<Widget>> ListAsync() =>
    _db.Set<Widget>().ToListAsync(); // …but we never filter on it
```

The `TenantId` column exists, so it *looks* multi-tenant. It isn't. Nothing enforces it.

2. Red — a test that proves the leak

Write the failing test first. Seed two tenants, put a widget in each, sign in as tenant A, and assert you can only see A's widget. Against the naive handler this test **fails** — and that failure is the whole point of the lesson.

```
// tests/Api.Tests/Tenancy/TenantIsolationTests.cs
[Fact]
public async Task Listing_widgets_never_returns_another_tenants_rows()
{
    var (tenantA, tenantB) = (await SeedTenantAsync(), await SeedTenantAsync());
    await SeedWidgetAsync(tenantA, "A's widget");
    await SeedWidgetAsync(tenantB, "B's widget");

    // Act as tenant A — CurrentTenant.TenantId == tenantA.Id
    using var scope = AsTenant(tenantA);
    var visible = await scope.Handler.ListAsync();

    Assert.Single(visible);
    Assert.Equal("A's widget", visible[0].Name); //   naive handler returns BOTH
}
```

> **TDD note.** We are not testing the handler's SQL. We're testing an *invariant of the platform*: "a tenant-scoped read is isolated." That invariant must hold no matter how careless the handler is — which is exactly why the fix belongs in the platform, not the handler.

3. Green — make isolation structural, not conventional

Three moves. First, mark the entity as tenant-owned by implementing a marker interface — this is the *opt-in* that the filter keys off:

```
// src/Core/Entities/ITenantScoped.cs
public interface ITenantScoped
{
    Guid TenantId { get; }
}

// src/Core/Entities/Widget.cs — now tenant-aware by type, not by discipline
public class Widget : ITenantScoped
{
    public Guid Id { get; set; }
    public Guid TenantId { get; set; }
    public string Name { get; set; } = "";
}
```

Second, resolve *who the current tenant is* from the authenticated request. That's a request-scoped abstraction the DbContext can depend on:

```
// src/Core/Abstractions/ICurrentTenant.cs
public interface ICurrentTenant
{
    Guid? TenantId { get; } // null when unauthenticated / tenant-less
}
```

The HTTP implementation reads the `tenant_id` claim off the JWT you issued in 2.2. In tests you swap in a trivial fake that returns whatever tenant you're acting as.

Third — the move that closes the leak for *every* entity at once — apply a global query filter in `OnModelCreating`, discovered by reflection so you can never forget to add it to a new entity:

```
// src/Infrastructure/Persistence/AppDbContext.cs
public Guid CurrentTenantId => _currentTenant.TenantId ?? Guid.Empty;

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);
    builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);

    // Every ITenantScoped entity is filtered to the current tenant by default,
    // so feature/domain queries can't forget to scope. Discovered, not hand-listed.
    foreach (var entityType in builder.Model.GetEntityTypes())
        if (typeof(ITenantScoped).IsAssignableFrom(entityType.ClrType))
            ApplyTenantFilterMethod.MakeGenericMethod(entityType.ClrType).Invoke(this,
[builder]);
}

private void ApplyTenantFilter<TEntity>(ModelBuilder builder) where TEntity : class,
ITenantScoped
    => builder.Entity<TEntity>().HasQueryFilter(e => e.TenantId == CurrentTenantId);
```

Now the *unchanged*, still-"careless" `ListAsync()` — `_db.Set<Widget>().ToListAsync()` — emits `WHERE tenant_id = @currentTenant` under the hood. Re-run the test: **green.** The handler never mentioned the tenant. The platform did.

4. Refactor & harden — fail closed, then lock it with a guardrail

Look hard at one line:

```
public Guid CurrentTenantId => _currentTenant.TenantId ?? Guid.Empty;
```

Why `Guid.Empty` and not "throw" or "return everything"? Because tenant ids are UUIDv7 — `Guid.Empty` is never a real row. So an **unauthenticated or tenant-less caller matches nothing** rather than *everything*. The failure mode of a missing tenant is "see no data," not "see all data." That is *fail-closed*, and it's a deliberate security decision, not an accident of defaulting. Add a test that pins it:

```
[Fact]
public async Task With_no_current_tenant_scoped_reads_return_nothing()
{
    await SeedWidgetAsync(await SeedTenantAsync(), "someone's widget");
    using var scope = AsNobody(); // ICurrentTenant.TenantId == null
    Assert.Empty(await scope.Handler.ListAsync()); // fail closed
}
```

Finally, make the invariant *enforceable by the build*. A filter you can bypass with `IgnoreQueryFilters()` is a filter a future slice will bypass. Add an architecture test (this is the recurring "guardrail ships with the feature" beat you'll see every part) that bans the escape hatch inside feature code:

```
// tests/Api.Tests/ArchitectureTests.cs
[Fact]
public void Feature_slices_never_call_IgnoreQueryFilters()
{
    var offenders = SourceFilesUnder("src/Api/Features")
        .Where(f => f.Contents.Contains("IgnoreQueryFilters"));
    Assert.Empty(offenders); // cross-tenant reads use the sanctioned QueryAllTenants()
seam
}
```

5. Run it

```
docker compose up -d # Postgres + Mailpit
dotnet test tests/Api.Tests --filter Tenancy
# 3 passing: isolation, fail-closed, and the arch guardrail
```

Optional: sign in via the running API as a user in tenant A, `GET /api/widgets`, and confirm B's rows are absent even though the SQL your handler wrote had no `WHERE` clause.

6. Why it's built this way

- **Structural beats conventional.** Isolation enforced by the *type system + one central

filter* can't be forgotten per-query. Isolation enforced by "remember to add `.Where`" will be forgotten — it's a matter of when. (ADR-003.)

- **Reflection over a hand-maintained list.** New entities are protected the moment they

implement `ITenantScoped`; there's no registration step to forget. (This is also why R11 builds the test-fixture reset list from the model, not by hand.)

- **Fail closed on purpose.** `?? Guid.Empty` turns "no tenant" into "no data." A system

that leaks when misconfigured is worse than one that shows an empty screen.

- **What deliberately *doesn't* implement `ITenantScoped`:** `User`, `TenantMembership`,

auth tokens — they're needed *before* a tenant is resolved, or span tenants. Recognizing which entities are cross-tenant is part of the skill this lesson teaches.

- **Reads are only half.** The filter scopes *reads*. A malicious or buggy *write* could

still stamp a foreign `TenantId`. That's the next lesson (2.6): the write-side `TenantStampingInterceptor` + the `EnterTenant` primitive.

7. Checkpoint

```
git tag lesson-2.5
```

You should now be able to: explain why a stored `TenantId` column is *not* multi-tenancy; add a global query filter keyed on a marker interface; justify a fail-closed default; and write a test that asserts a platform invariant rather than a single method's behavior.

Next: *Lesson 2.6 — Write-side tenancy*, where we stop trusting handlers to set `TenantId` correctly on the way *in*.